

RemoteLab

Mobile Browser Access to an Authorized Kali Linux Lab

LinkedIn Project Walkthrough - Screenshot-by-Screenshot

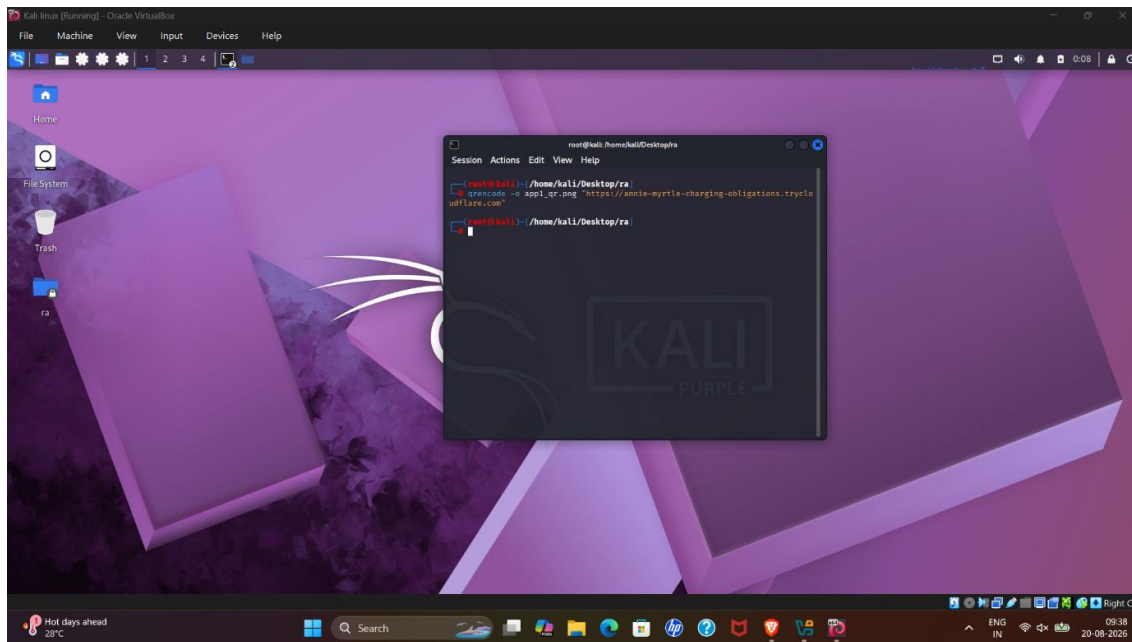
Project Overview

RemoteLab is a proof-of-concept project that demonstrates how a mobile browser can reach a controlled Kali Linux lab through a web interface. The workflow used in the demonstration is:

QR Code -> Mobile Browser -> Cloudflare Tunnel -> Flask Application -> Kali Linux Lab

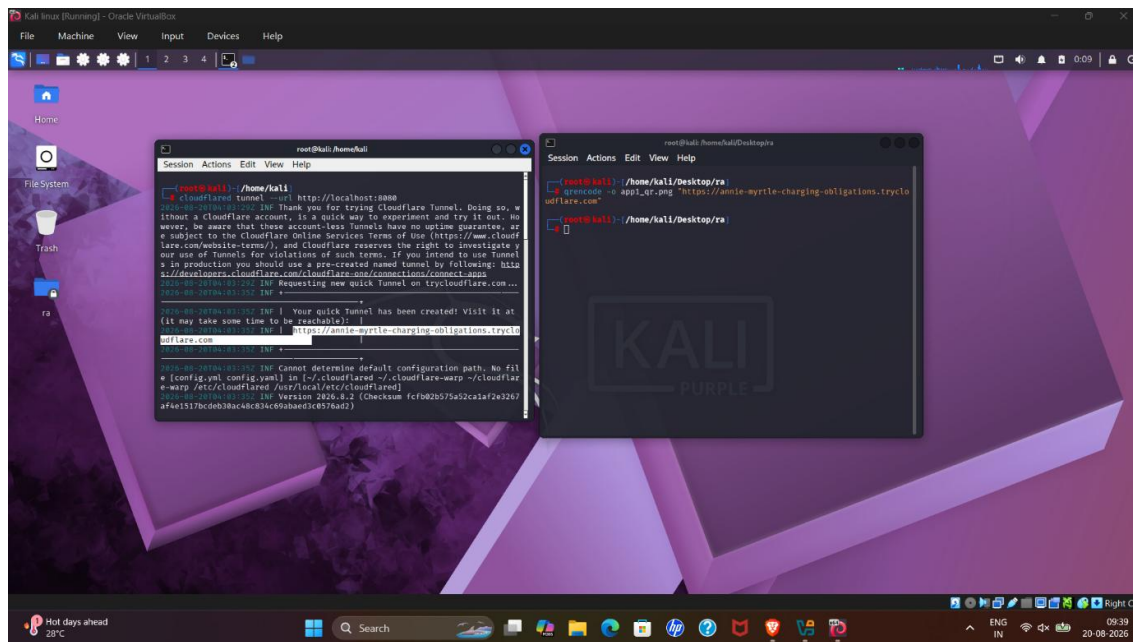
The screenshots below document the development and successful test of the proof of concept. The public portfolio version should use the hardened code supplied with the GitHub package rather than the original unrestricted command-execution prototype.

Screenshot 1 - Starting the Kali-side project



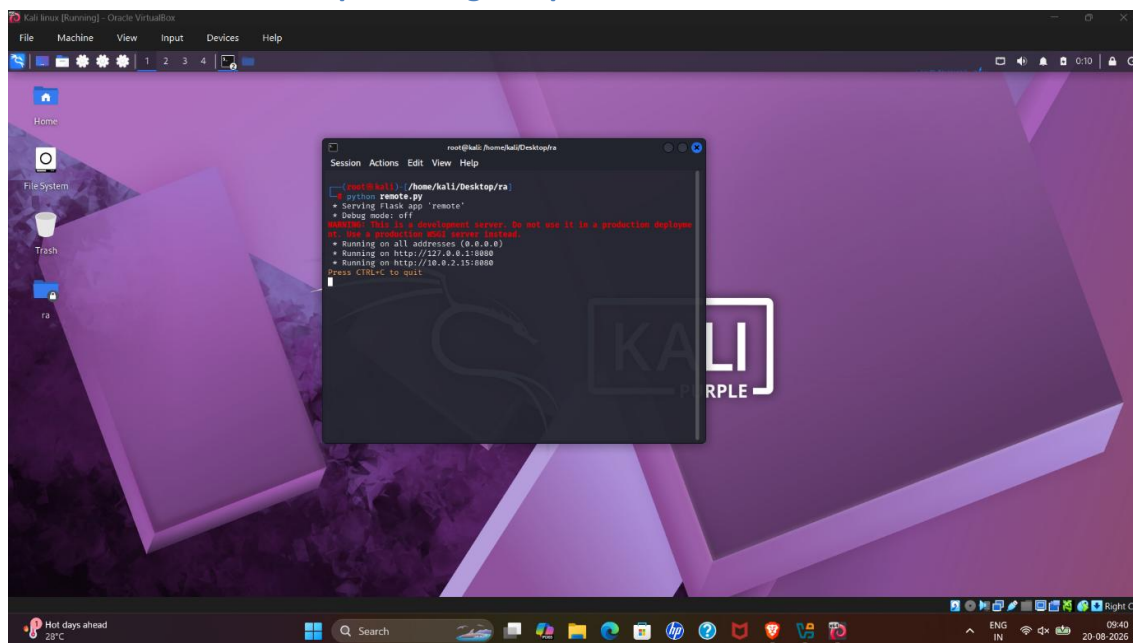
This screenshot shows the Kali Linux desktop with a terminal opened for the project. It establishes the lab environment where the Flask application and supporting components are being run. For LinkedIn, this works as the starting point of the build rather than as proof of the final result.

Screenshot 2 - Running the service and tunnel workflow



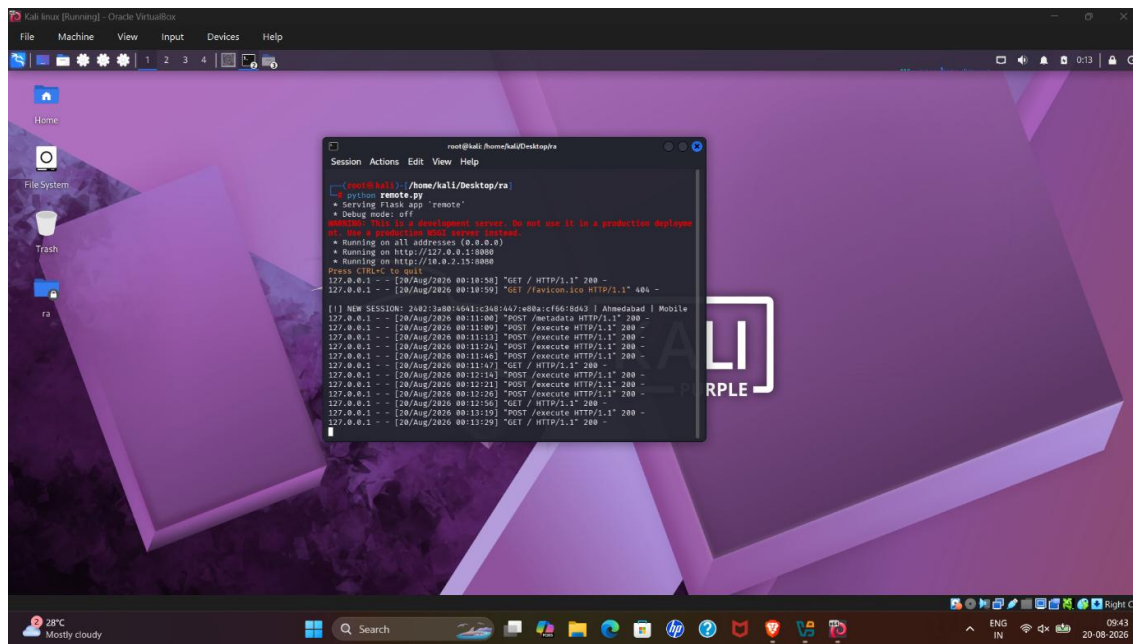
The second screenshot shows additional terminal activity while the project is being launched and made reachable. It represents the server/tunnel stage of the architecture: the Kali-hosted application is prepared so that a mobile browser can reach it remotely.

Screenshot 3 - Kali terminal output during setup



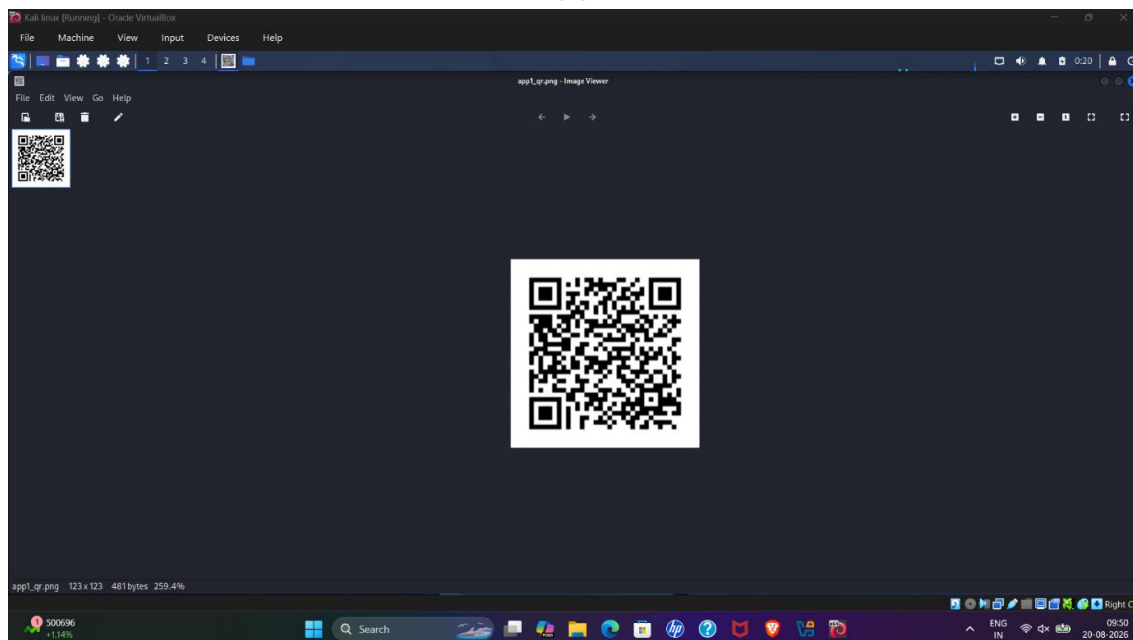
This screenshot captures command-line output from the Kali environment during the setup/test process. It demonstrates that the project was built and tested directly in a Linux lab rather than being only a UI mock-up.

Screenshot 4 - Network/service output



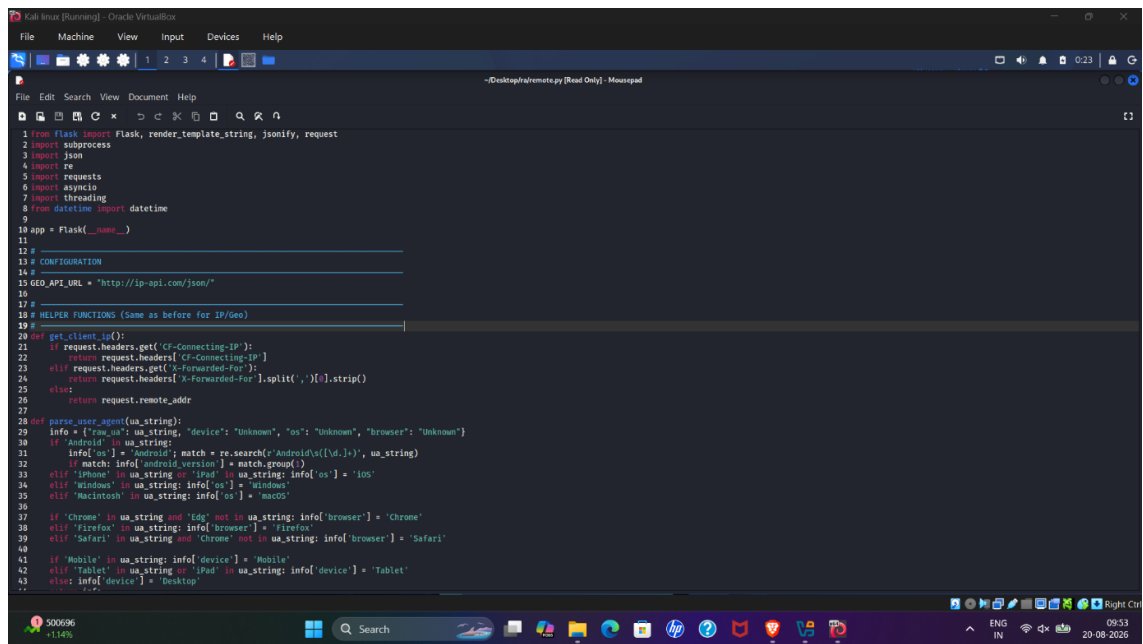
The fourth desktop screenshot shows more detailed terminal output associated with the running environment. For a public post, network details in screenshots should be treated carefully: IP addresses, MAC addresses, tunnel identifiers, hostnames, or other identifiers should be redacted when they are not necessary to explain the project.

Screenshot 5 - QR code used as the mobile entry point



This screenshot shows the QR code generated for the project. Scanning the QR code provides a convenient entry point to the web interface. The QR code itself should not be published if it still resolves to an active or reusable tunnel URL; use a disabled/demo QR code for a public portfolio.

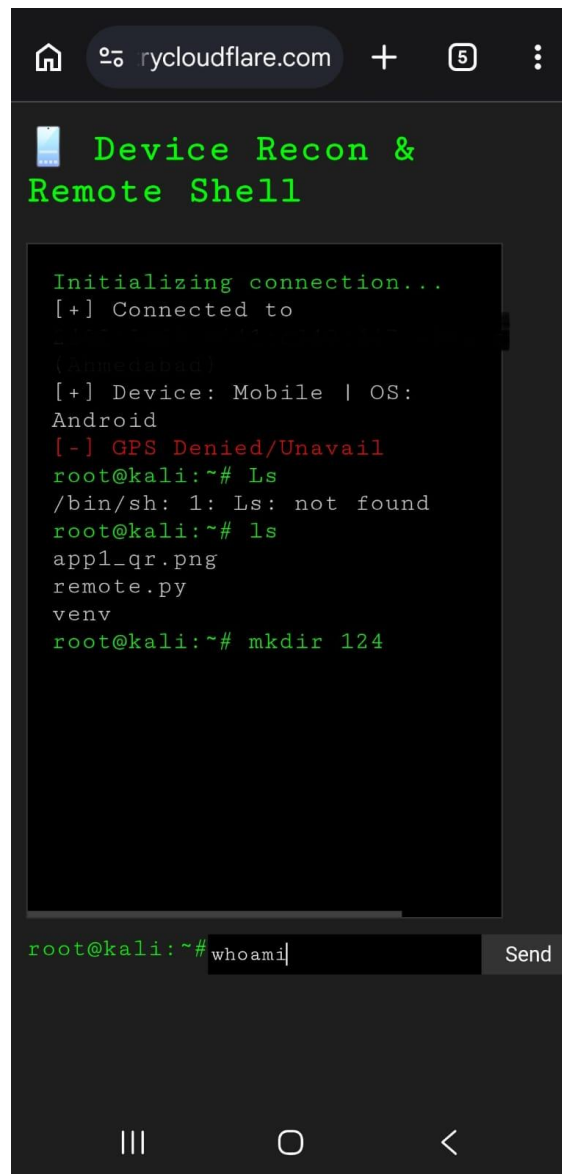
Screenshot 6 - Python/Flask implementation



```
1 from flask import Flask, render_template_string, jsonify, request
2 import subprocess
3 import json
4 import re
5 import requests
6 import asyncio
7 import threading
8 from datetime import datetime
9
10 app = Flask(__name__)
11
12 #
13 # CONFIGURATION
14 #
15 GEO_API_URL = "http://ip-api.com/json/"
16
17 #
18 # HELPER FUNCTIONS (Same as before for IP/Geo)
19 #
20 def get_client_ip():
21     if request.headers.get('CF-Connecting-IP'):
22         return request.headers['CF-Connecting-IP']
23     elif request.headers.get('X-Forwarded-For'):
24         return request.headers['X-Forwarded-For'].split(',')[0].strip()
25     else:
26         return request.remote_addr
27
28 def parse_user_agent(ua_string):
29     info = {'raw_ua': ua_string, 'device': 'Unknown', 'os': 'Unknown', 'browser': 'Unknown'}
30     if 'Android' in ua_string:
31         info['os'] = 'Android'; match = re.search(r'Android\((\d.\d+)\)', ua_string)
32         if match: info['android_version'] = match.group(1)
33     elif 'iPhone' in ua_string or 'iPad' in ua_string: info['os'] = 'iOS'
34     elif 'Windows' in ua_string: info['os'] = 'Windows'
35     elif 'Macintosh' in ua_string: info['os'] = 'macOS'
36
37     if 'Chrome' in ua_string and 'Edg' not in ua_string: info['browser'] = 'Chrome'
38     elif 'Firefox' in ua_string: info['browser'] = 'Firefox'
39     elif 'Safari' in ua_string and 'Chrome' not in ua_string: info['browser'] = 'Safari'
40
41     if 'Mobile' in ua_string: info['device'] = 'Mobile'
42     elif 'Tablet' in ua_string or 'iPad' in ua_string: info['device'] = 'Tablet'
43     else: info['device'] = 'Desktop'
44     return info
```

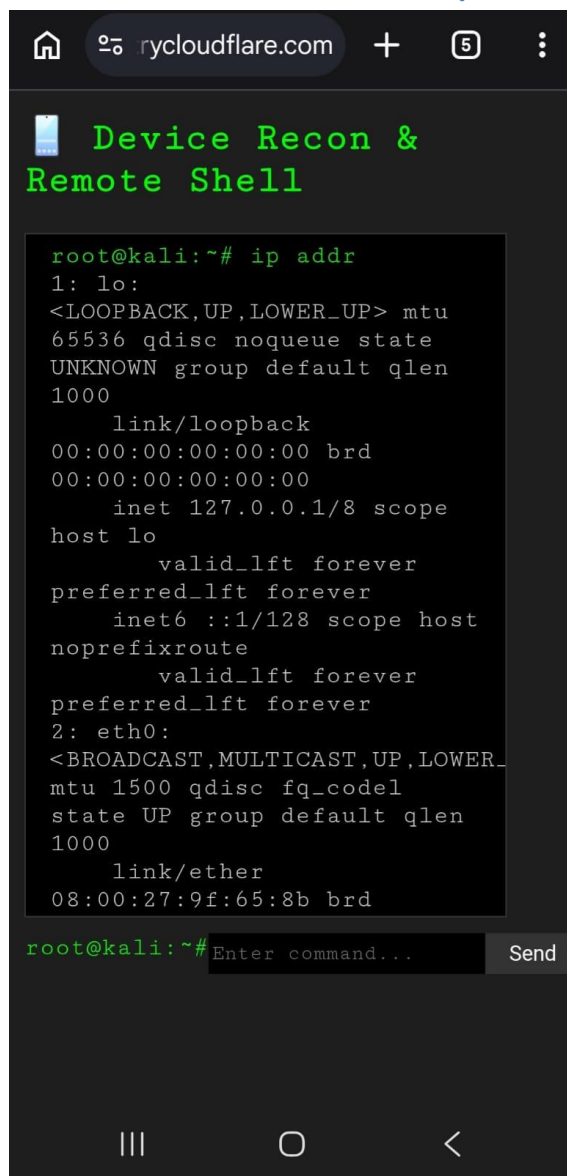
This screenshot shows the Python source used to build the web interface and backend. The original proof of concept included device metadata collection and an unrestricted command endpoint. For the public GitHub release, the safer version removes GPS/IP-geolocation collection and replaces arbitrary shell execution with authenticated, allow-listed commands.

Screenshot 7 - Successful mobile connection and command execution



This is one of the strongest demonstration screenshots. The phone browser has reached the RemoteLab interface and displays the terminal-style UI. Commands such as `ls` and `mkdir` are shown being sent to the Kali environment, demonstrating end-to-end communication from the mobile browser to the lab backend. The original interface displays `root@kali`, but the public version should run as an unprivileged user.

Screenshot 8 - Retrieving Kali network information from the phone



The final screenshot shows `ip addr` being executed from the mobile interface and the Kali network-interface output being returned to the phone. This proves that the browser request reached the Kali-side backend and that command output was successfully returned to the mobile UI. Before posting this image publicly, redact MAC addresses, IP addresses, tunnel URLs, and any other network identifiers that are not needed for the demonstration.

What I Learned

- How a Flask application can expose a browser-based interface to a Linux lab.
- How a tunnel can bridge a locally hosted application to a mobile browser.
- How QR codes can be used as a convenient access mechanism for a web endpoint.
- Why unrestricted remote command execution is a major security boundary.
- Why a portfolio release should minimize telemetry and avoid collecting GPS or unnecessary device information.
- Why screenshots themselves require a privacy review before being posted publicly.

Security Improvements Before Public Release

The first proof of concept prioritized getting the end-to-end workflow working. During review, the remote command endpoint and privacy-sensitive telemetry were identified as risks. The GitHub portfolio version therefore uses token-based access control, an explicit command allow-list, shell=False, no GPS collection, no IP geolocation, and no requirement to run the application as root.

Suggested LinkedIn Caption

Built RemoteLab - a mobile browser interface for accessing my authorized Kali Linux lab.

The workflow: QR Code -> Mobile Browser -> Cloudflare Tunnel -> Flask -> Kali Linux.

The first proof of concept successfully let me send commands from my phone and receive output from the Kali lab. Reviewing the design also exposed an important lesson: convenient remote command execution can quickly become a security boundary. Before preparing the public GitHub version, I removed GPS/IP-geolocation collection, stopped relying on root access, added access control, and replaced unrestricted shell execution with an allow-list.

The project helped me practice Linux, Flask, networking, tunneling, remote administration, and - most importantly - reviewing my own design from a security perspective.

#CyberSecurity #KaliLinux #Python #Flask #Linux #Networking #SecurityEngineering

Privacy Check Before Posting

- Do not publish an active Cloudflare tunnel URL or reusable QR code.
- Redact public/private IP addresses and MAC addresses that you do not want exposed.
- Remove GPS coordinates, city/location information, phone numbers, email addresses, tokens, cookies, and passwords.
- Check the phone status bar, browser tabs, account icons, notifications, and terminal username/hostname.
- Strip image metadata/EXIF from screenshots before publishing.